

TRAVAUX PRATIQUES DE PROGRAMMATION LOGIQUE

SEANCE DE TP PROLOG N°1 : LES BASES

1^{ère} partie

Soit un ensemble de personnes et de relations décrits en Prolog par les paquets de clauses suivants :

```
% Faits
personne(edouard, homme, 72, retraite(commandant(armee_de_terre)), nice).
personne(agathe, femme, 58, musicienne(blues), nice).
personne(gaston, homme, 65, retraite(chauffeur(semvat)), balma).
personne(simone, femme, 65, retraite(cableur(airbus)), balma).
personne(georges, homme, 28, animateur(club_med), toulouse).
personne(marie_cecile, femme, 35, clerc_de_notaire, toulouse).
personne(octave, homme, 40, huissier, nice).
personne(elton, homme, 10, ecolier, toulouse).
personne(vanessa, femme, 5, ecolier, toulouse).

parent(edouard, marie_cecile).
parent(agathe, marie_cecile).
parent(edouard, octave).
parent(agathe, octave).
parent(gaston, georges).
parent(simone, georges).
parent(georges, elton).
parent(georges, vanessa).
parent(marie_cecile, elton).
parent(marie_cecile, vanessa).

% Règles
pere(Pere, Enfant) :-
    personne(Pere, homme, Age, Profession, Ville),
    parent(Pere, Enfant).

fille(Fille, Parent) :-
    personne(Fille, femme, _, _, _),
    parent(Parent, Fille).
```

1. Sauvegarder ce programme source dans le fichier **tp1_1.ecl** (pour ECLⁱPS^e) ou **tp1_1.pl** (pour Gnu Prolog ou Swi Prolog). Lancer Prolog, puis compiler le fichier source avec la commande :

```
?- consult(tp1_1).           ou      ?- [tp1_1].
```

2. Indiquer les réponses aux questions suivantes dans le compte-rendu :

- Pourquoi la clause définissant le prédicat **pere/2** est-elle responsable d'un warning ?
- Quelle solution est apportée dans la clause qui définit **fille/2** ? (que signifie le symbole '_' en Prolog ?)
- Quelles requêtes Prolog doit-on formuler pour déterminer :
 - les personnes domiciliées à Nice ?
 - les personnes qui habitent dans la même ville que leurs deux parents ?
- (joindre les réponses aux requêtes dans le compte-rendu)

3. Compléter le programme en ajoutant les règles qui permettent de définir les prédicats **mere/2** et **fils/2**.

4. On soumet la requête suivante :

```
?- parent(P,E) , personne(E,_,_,ecolier,_).
```

Quel est le sens de cette requête ?

5. On désire ajouter une nouvelle règle définissant la relation **parent_d_ecolier(P)** vraie ssi P a un enfant écolier. On souhaite obtenir tous les noms des personnes qui ont un enfant écolier mais une seule fois chaque nom (on veut une seule réponse P même si P est une personne ayant plusieurs enfants écoliers). Pour cela on peut faire appel au prédicat prédéfini de contrôle **once(Goals)** qui réussit en prenant la première solution possible pour satisfaire **Goals**, exemple :

```
?- once( (personne(P,_, Age,_,_) , Age > 60) ) .
P = edouard
Age = 72 ;
No (0.00s cpu)
```

- Tester la règle que vous avez proposée en vérifiant qu'elle fournit bien deux réponses :

```
?- parent_ecolier(P) .
P = georges ;
P = marie_cecile ;
no (0.00s cpu)
```

2^{ème} partie

Le menu d'un restaurant diététique a été représenté par les clauses Prolog suivantes :

```
% LES ENTREES
hors_d_oeuvre(artichauts_melanie).
hors_d_oeuvre(truffes_sous_le_sel).
hors_d_oeuvre(cresson_oeuf_poche).

% LES VIANDES
viande(grillade_de_boeuf).
viande(poulet_au_tilleul).

% LES POISSONS
poisson(bar_aux_algues).
poisson(truite_meuniere).

% LES DESSERTS
dessert(sorbet_aux_poires).
dessert(fraises_chantilly).
dessert(melon_surprise).
```

```

% LES BOISSONS (25 cl)
boisson(eau_minerale(hepar)).
boisson(vin_rouge(morgon)).
boisson(biere(gueuze)).

% VALEURS CALORIFIQUES
calories(artichauts_melanie, 150).
calories(truffes_sous_le_sel, 212).
calories(cresson_oeuf_poche, 202).
calories(grillade_de_boeuf, 532).
calories(poulet_au_tilleul, 400).
calories(bar_aux_algues, 292).
calories(truite_meuniere, 254).
calories(sorbet_aux_poires, 223).
calories(fraises_chantilly, 289).
calories(melon_surprise, 122).
calories(eau_minerale(hepar), 0).
calories(vin_rouge(morgon), 175).
calories(biere(gueuze), 250).

```

Sauvegarder ce programme source dans le fichier **tp1_2.ec1** (pour ECLⁱPS^e) ou **tp1_2.pl** (pour [Gnu,Swi]-Prolog)

1. Compléter le programme en ajoutant les règles qui permettent de définir les relations :

plat(P) vraie si P est une viande ou un poisson.

repas_leger(R) vraie si R est une liste [Entree,Plat,Dessert,Boisson] représentant un repas de moins de 600 calories.

2. Proposer une deuxième version de la relation **repas_leger(R,X,L)** vraie si R est un repas représentant **X** calories, **X** étant inférieur à une certaine limite **L** donnée. Existe-t-il des repas de moins de 550 calories ?

3^{ème} partie : Récursions arithmétiques

Factorielle

On donne le programme suivant qui associe à un nombre donné N, la valeur N! (factorielle(N)) :

```

fact(0,1) :- !.                                % factorielle(0) = 1
fact(N,F) :-                                    % factorielle(N) = N*factorielle(N-1)
    N1 is N-1,
    fact(N1,F1),
    F is N*F1.

```

Sauver ce programme dans le fichier **tp1_3** Joindre en commentaires les réponses aux questions suivantes :

1. Expliquer la présence du coupe-choix (!) dans la 1^{ère} clause du programme.
2. Concevoir une nouvelle version du programme qui évite l'usage du coupe-choix.
3. Comparer les deux versions (avantages/ inconvénients) sur la requête :

```
?- fact(-10,F).
```

Somme d'une liste d'entiers

1. Définir en 2 clauses (clause d'arrêt et clause de récursion) le prédicat **somme (+L, ?S)**. A l'appel, le premier argument est nécessairement connu (instancié) ; le second, qui représente le résultat du calcul peut être libre ou instancié ; exemples :

```
?- somme([1,2,3,4], S).
S = 10;
No

?- somme([1,2,3,4], 9).
Yes
```

2. On donne une autre version de ce prédicat :

```
somme2([], S, S).
somme2([X|L], I, S) :-
    J is X+I,
    somme2(L, J, S).
```

Quelles sont les réponses aux requêtes suivantes :

```
?- somme2([1,2,3,4], 0, S).
?- somme2([1,2,3,4], 0, 9).
?- somme2([1,2,3,4], 123, S).
?- somme2([1,2,3,4], 0, 10).
```

3. Quelles différences existe-t-il entre les algorithmes sous-jacents aux deux versions ? Quels sont les avantages et inconvénients respectifs de chaque version ?

Suite de Fibonacci

La définition mathématique de la suite de Fibonacci est donnée par :

$$\text{fibonacci}(n) = 0 \text{ si } n=0 ; \text{ fibonacci}(n) = 1 \text{ si } n=1 ; \text{ fibonacci}(n) = \text{fibonacci}(n-1) + \text{fibonacci}(n-2) \quad \forall n \geq 2$$

1. En s'inspirant du programme **fact/2** écrire les clauses du prédicat **fibonacci/2** correspondant aux définitions données ci-dessus. Ce programme réussit-il pour la requête `?- fibonacci(30, F).` ? Pourquoi ?
2. Donner une nouvelle version, **fibonacciplus/3** qui associe à un nombre donné n **deux** termes consécutifs de la suite, fibonacci(n) et fibonacci(n-1). Ce programme réussit-il pour la requête `?- fibonacciplus(56000, F, _).`

SEANCE DE TP PROLOG N°2 : RECURSIONS SUR DES LISTES

1^{ère} partie : Palindrome

1. Définir le prédicat **palindrome(P)** vrai si P est la même liste, lue de gauche à droite ou de droite à gauche, ex :

```
?- palindrome([]) .
Yes
?- palindrome([k,a,y,a,k]) .
Yes
?- palindrome([X,a,v,a,l]) .
X = l
Yes
?- palindrome([X,X]) .
X = X
Yes
?- palindrome([1,2,3]) .
No
```

2^{ème} partie : Dispatch et QuickSort

1. Définir la relation **dispatch(X, L, Infeg, Sup)** qui répartit la liste **L** en 2 sous-listes : **Infeg** est la sous-liste des éléments de **L** inférieurs ou égaux à **X** ; **Sup** est la sous-liste des éléments de **L** supérieurs à **X**.
2. Réutiliser cette relation pour définir **quicksort(Unsorted, Sorted)** qui détermine la liste **Sorted** triée par ordre croissant des éléments de la liste **Unsorted**, ex :

```
?- quicksort([1,10,2,9,3,8,4,7,5,6], Sorted) .
Sorted = [1,2,3,4,5,6,7,8,9,10]
Yes
```

3. En ajoutant un paramètre supplémentaire, donner une deuxième version qui n'appelle pas **append/3**.

```
?- quicksort2([1,10,2,9,3,8,4,7,5,6], [], Sorted) .
Sorted = [1,2,3,4,5,6,7,8,9,10]
Yes
```

Quel est l'intérêt de ne pas appeler **append/3** dans la 2^{ème} version ?

3^{ème} partie : Décomposition d'une liste

1. Définir la relation **a_droite(X,L,D)** vraie si D est la liste des éléments se trouvant à droite de l'élément X dans la liste L ; même question pour **a_gauche(X,L,G)**.
2. On désire fusionner les deux relations précédentes en une seule sans pour autant y faire appel : **separer(X,L,G,D)** découpe une liste L en deux listes G et D se trouvant de part et d'autre de la même occurrence de X dans L. Ecrire ce programme en respectant la consigne (ne faire aucun appel ni à **a_droite/3** ni à **a_gauche/3**). Vérifier son bon fonctionnement sur les requêtes suivantes :

```
?- separer(2, [1,2,3,4,2,5], G, D) .

?- separer(3, L, [1,2], [4,5]) .
```

On convient de représenter un graphe $G = (X, U)$ fini, orienté et sans cycles, par une clause Prolog liant un identifiant de graphe G à sa liste de sommets X et sa liste d'arcs U ; par convention, chaque arc est représenté par un terme [Origine, Extrémité]. Le graphe de nom $g1$ représenté ci-dessous comporte 6 sommets et 6 arcs :

```
graphe(g1, [1,2,3,4,5,6], [[1,2],[1,3],[2,4],[3,4],[4,5],[4,6]]).
```

1. En utilisant **member/2** et **graphe/3**, définir la relation, **arc(G,O,E)** vraie s'il existe un arc allant d'un sommet O à un sommet E sur le graphe de nom G .
2. En utilisant **arc(G,O,E)**, définir récursivement la relation **existe_chemin(G,O,E)** vraie s'il existe sur le graphe G , un chemin joignant le sommet O au sommet E .
3. En s'inspirant de **existe_chemin/3** auquel on adjoint un 4^{ème} argument, définir la relation **chemin(G,O,E,Ch)** qui construit un chemin $Ch = [O, X_1], [X_1, X_2], \dots, [X_n, E]$, reliant O à E dans G .
4. Quelles sont les différents modes d'utilisation possibles des prédicats définis précédemment : quels arguments peuvent/doivent être instanciés à l'appel ? Donner des exemples.
5. Les graphes étudiés maintenant peuvent contenir des circuits, comme le graphe $g2$:

```
graphe(g2, [1,2,3,4,5,6], [[1,2],[1,3],[2,3],[2,4],[3,4],[4,1],[4,5],[4,6]]).
```

Que se passe-t-il si on utilise la relation **existe_chemin/3** ou **chemin/3** sur ce graphe ?

6. Définir la relation **chemin_sans_circuit(G, O, E, Ch, Memo)** qui construit un chemin Ch (liste de sommets ou liste d'arcs, au choix), allant de O à E **sans jamais quitter deux fois le même sommet**. L'argument **Memo** sert à collecter les sommets déjà quittés, afin d'éviter de parcourir les arcs qui aboutissent à un sommet qu'on a déjà quitté. Préciser la valeur initiale de **Memo** à l'appel : $[]$ ou $[O]$? Indiquer clairement à l'aide d'exemples toutes les utilisations possibles intéressantes de ce prédicat.

ANNEXE : DIFFERENTES VERSIONS DE PROLOG

Les différentes versions de Prolog disponibles en TP

- 3 versions de Prolog sont disponibles sous Linux :

- ECLiPS^e Prolog

Lancement dans un GUI :	tkeclipse &
Lancement dans une console :	/usr/local/eclipse.prolog/bin/i386_linux/eclipse &
Suffixe des fichiers source	.ecl

- Gnu Prolog

Lancement dans une console :	gprolog &
Suffixe des fichiers source	.pl

- Swi Prolog

Lancement dans une console :	swipl &
Suffixe des fichiers source	.pl